

Ricerca binaria (*Binary Search*)

Algorithm 1: Search the value v among the n ordered elements of S

```
1 function BINARYSEARCH( $S, v, first, last$ )
2   if  $first > last$  then
3     return 0;
4    $m \leftarrow \lfloor \frac{first+last}{2} \rfloor$ ;
5   if  $S[m] = v$  then
6     return  $m$ ;
7   else
8     if  $S[m] > v$  then
9       return binarySearch( $S, v, first, m - 1$ );
10    else
11      return binarySearch( $S, v, m + 1, last$ );
```

Il cuore dell'algoritmo sta nella "divisione" dell'array, tutto in maniera ricorsiva. Se seguissimo "ciclo per ciclo" notiamo come prendiamo sempre il valore centrale, ovvero m , e poi analizziamo il sotto vettore a "destra" di m , o alla sua sinistra, a seconda del valore v che cerchiamo.

Attenzione, i parametri $first$ e $last$ rappresentano gli **indici** della nostra struttura, non gli elementi. Per questo motivo la condizione $first > last$ ha il solo scopo di dirci se i nostri indici si sono "scambiati", e ciò avviene solo se **non** è stato trovato v .

Prendiamo come esempio un array del tipo $[1, 2, 3, 4, 9, 11]$, e poniamo l'elemento da cercare come $v = 9$; seguiamo ogni passaggio.

Prima iterazione: ignoriamo la prima condizione per quanto detto nel paragrafo prima e notiamo subito che la prima istruzione sarà:

$$m = \frac{first + last}{2} \rightarrow m = \frac{0 + 5}{2} \rightarrow m \approx 3$$

Seguita dalla condizione $S[3] = 9$ che sarà falsa in quanto l'elemento alla posizione $S[3] = 4$.

Entriamo ora nel punto dell'algoritmo dove "scendiamo" nel sottoarray generato a partire da m , che in questo caso sarà il sottoarray di destra, ovvero la struttura con elementi $[9, 11]$. Ripeteremo quindi questa iterazione usando i nuovi parametri $(S, v = 9, first = m + 1, last)$.

Seconda iterazione: ignoriamo di nuovo la prima condizione, e calcoliamo il nuovo m :

$$m = \frac{first + last}{2} \rightarrow m = \frac{4 + 5}{2} \rightarrow m \approx 4$$

E in questo caso $S[m] = v$ restituisce *true* quindi restituiremo l'indice appena trovato $m = 4$.

Costo computazionale

Il caso migliore di questo algoritmo è ovviamente un $O(1)$ poichè se l'elemento da trovare (v) è esattamente al centro $S[m] = v$ darà subito *true*.

Il caso peggiore invece è un pochino più complesso. Innanzitutto questo caso non è altro che lo scenario dove **non troviamo** l'elemento v o quello in cui v è posto come estremo dell'array (primo o ultimo elemento).

Quello che farà il nostro algoritmo sarà quindi dimezzare l'array ad ogni iterazione:

- $input_1 = n/2$
- $input_2 = n/2^2$
- $input_3 = n/2^3$
- $input_i = n/2^i$

Nel caso peggiore in cui l'elemento è agli estremi dell'array arriveremo all'ultima iterazione dove il sottoarray sarà di un solo elemento, vale a dire che avremo un input del tipo:

$$n/2^i = 1$$

Adesso ci basta solo applicare della matematica per trovare che:

$$n = 2^i \rightarrow i = \log_2 n$$

Il caso peggiore del **binary search** è asintoticamente pari a $O(\log n)$